

任务1.1 代码实现：

损失函数

1. MSE Loss

公式：

$$L = \frac{1}{N} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1)$$

2. L1 Loss

公式：

$$L = \frac{1}{N} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2)$$

3. Charbonnier Loss

公式：

$$L = \frac{1}{N} \sum_{i=1}^n \sqrt{(y_i - \hat{y}_i)^2 + \epsilon} \quad (3)$$

4. MSE L1 Loss

公式：

$$L = \frac{1}{N} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \frac{1}{N} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4)$$

其中 α 对应输入中的 `l1_weight` .

5. MSE Edge Loss

公式：

$$L = \frac{1}{N} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left[\frac{1}{N_x} \sum_{i=1}^{n_x} (\Delta x_i)^2 + \frac{1}{N_y} \sum_{i=1}^{n_y} (\Delta y_i)^2 \right] \quad (5)$$

其中：

$$\Delta x_i = (P_{i,j+1,c} - P_{i,j,c}) - (T_{i,j+1,c} - T_{i,j,c}) \quad (6)$$

$$\Delta y_i = (P_{i+1,j,c} - P_{i,j,c}) - (T_{i+1,j,c} - T_{i,j,c}) \quad (7)$$

优化器

1. SGD:

公式：

$$\theta_{t+1} = \theta_t - \eta g_t \quad (8)$$

其中 η 是学习率, g_t 是当前参数的梯度, 也就是 `loss` 增长最快的方向。

2. SGD + Momentum

公式:

$$v_{t+1} = \mu v_t + g_t, \quad \theta_{t+1} = \theta_t - \eta v_{t+1} \quad (9)$$

引入了动量、速度的概念, 其中 μ 一般设为0.9

3. Adam

公式:

一阶动量:

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) g_t \quad (10)$$

二阶动量:

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) g_t^2 \quad (11)$$

偏差修正:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (12)$$

更新公式:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (13)$$

在 `Momentum` 的基础上引入了二阶动量, 且不同参数的更新幅度会受 $\frac{1}{\sqrt{\hat{v}_t} + \epsilon}$ 的影响。

4. AdamW

更新公式:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda \theta_t \quad (14)$$

采用正则化的思想, 通过在 `Adam` 的基础上添加 $-\eta \lambda \theta_t$ 来防止 `Adam` 过拟合。

5. Muon

更新公式:

$$b_{t+1} = \mu b_t + g_t, \quad \tilde{g}_{t+1} = \text{Orth}(b_{t+1}), \quad \theta_{t+1} = \theta_t - \eta \tilde{g}_{t+1} \quad (15)$$

其中 b_t 表示动量缓冲区, μ 表示动量系数, $\text{Orth}(\cdot)$ 表示对更新方向进行近似正交化处理。

`Muon` 的核心思想是在 `Momentum` 的基础上, 不直接使用累积后的梯度进行更新, 而是先对更新方向做一次结构化归一化和正交化, 从而让参数更新方向更加稳定, 避免某些方向的更新幅度过大。

初始化

Grid 网格初始化

根据得到的 `num_gaussians` 去开方得到近似的行数和列数，然后生成相应个数的 `x` 和 `y` 点，`center_init` 最后就是 `x` 和 `y` 的两个数列中元素逐个组合的结果。

其余参数沿用随机初始化的代码。

ImageSample 初始化

其他参数统一沿用随机初始化的代码，然后利用随机初始化的 `center_init` 去从原始图像中找到对应的点的颜色，然后作为该高斯点的颜色。

学习率调度器

本项目中学习率调度器返回的都不是学习率本身，而是 `base_lr` 的系数。真正的学习率为：

$$lr = base_lr \times \lambda \quad (16)$$

但后续为了与传统公式一致，仍用 η_t 表示学习率调度器的返回结果。

1. cosine

公式：

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos \frac{t\pi}{T}) \quad (17)$$

其中由于 $\eta_{min} = 0, \eta_{max} = 1$ ，因此公式化简为：

$$\eta_t = 0.5 \times (1 + \cos \frac{t\pi}{T}) \quad (18)$$

2. warmup_cosine

公式：

$$\eta_t = \begin{cases} \frac{step}{warmStep}, & step \leq warmStep \\ cosine(step - warmStep, totalSteps - warmStep), & else \end{cases} \quad (19)$$

其中在本代码中 `warmStep` 设置为10。

该调度器在前 `warmStep` 步会从 η_{min} 逐步升至 η_{max} ，然后后续同 `cosine` 调度器一样进行学习率下降。

3. step_decay

公式：

$$\eta_t = \gamma^{\lfloor \frac{step-1}{changeStep} \rfloor} \quad (20)$$

其中在本代码中 `changeStep` 设为5，`gamma` 设为0.8。

该调度器会在每 `changeStep` 步后，是学习率变为之前的 `gamma` 倍。

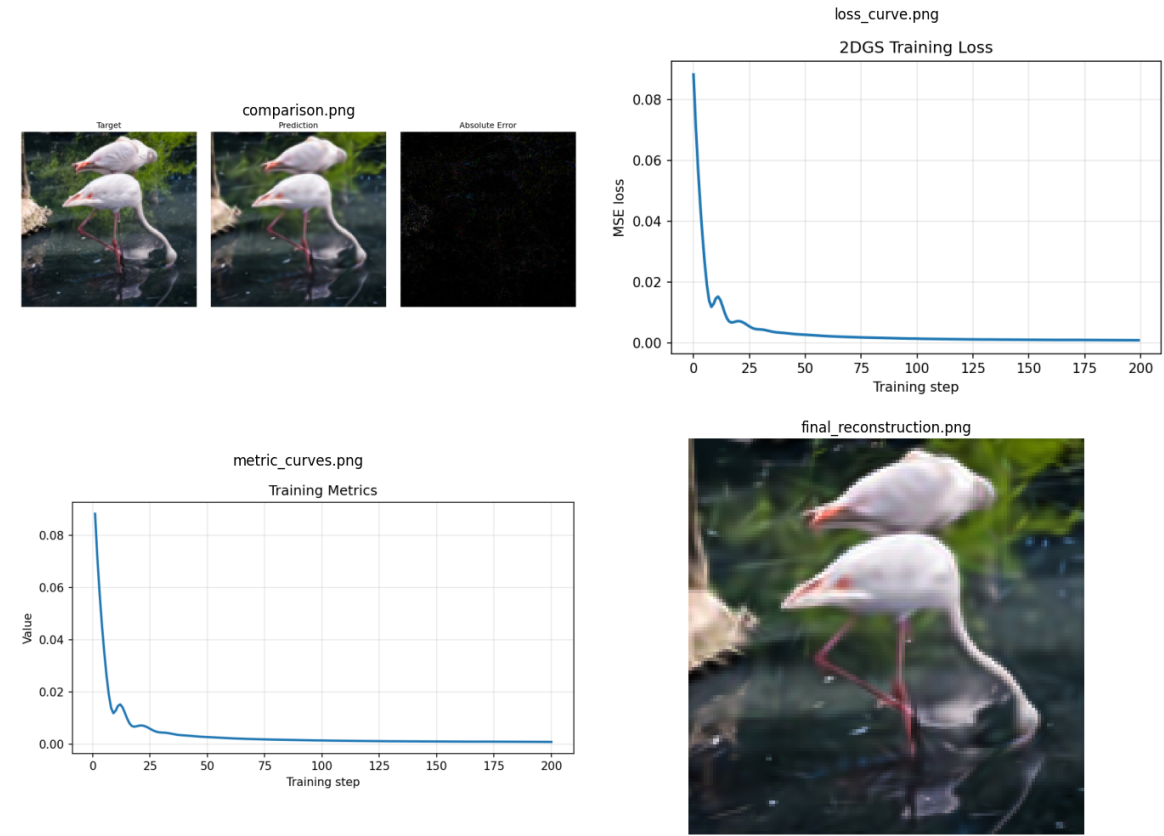
任务 1.2 消融实验报告

消融实验结果

初始模块配置：

项目	图像大小	高斯数量	步数	Loss	Optimizer	Scheduler	Initializer	各向异性	Alpha	随机种子
默认配置	128×128	1000	200	mse	torch_adam	constant	random	True	True	42

基准实验结果：



最终测试结果汇总：

消融实验/指标		PSNR	MSE	MAE
基准实验		30.5979	0.00087138	0.01989225
loss	l1_loss	29.7488	0.00105956	0.01960821
	charbonnier_loss	30.553	0.00088044	0.01844228
	mse_l1_loss	30.2017	0.00095462	0.01942236
	mse_edge_loss	30.7402	0.0008433	0.01955922
初始化策略	grid	30.8039	0.00083102	0.01954995
	image_sample	30.336	0.00092555	0.02060915
优化器	sgd	10.5858	0.08738101	0.20758496
	momentum	10.9435	0.08047271	0.19918978
	adam	30.6168	0.00086761	0.01985523
	adamw	30.1309	0.0009703	0.02077659
	muon	20.3466	0.00923287	0.07410043
模型设计	False-False	26.3716	0.00230592	0.03556243
	True-False	28.1179	0.00154243	0.0295183
	False-True	27.5295	0.00176626	0.02747365
学习率调度器	cosine	28.9237	0.00128123	0.02374906
	warmup_cosine	28.4568	0.00142667	0.02496918
	step_decay	24.6908	0.00339559	0.03815757

评价指标

1. MSE 均方误差

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2 \tag{21}$$

特点:

- 对大误差有更加严重的惩罚

2. MAE 平方绝对误差

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - y_i| \tag{22}$$

特点:

- 所有误差线性处理，但对严重坏点不够敏感

3. PSNR 峰值信噪比

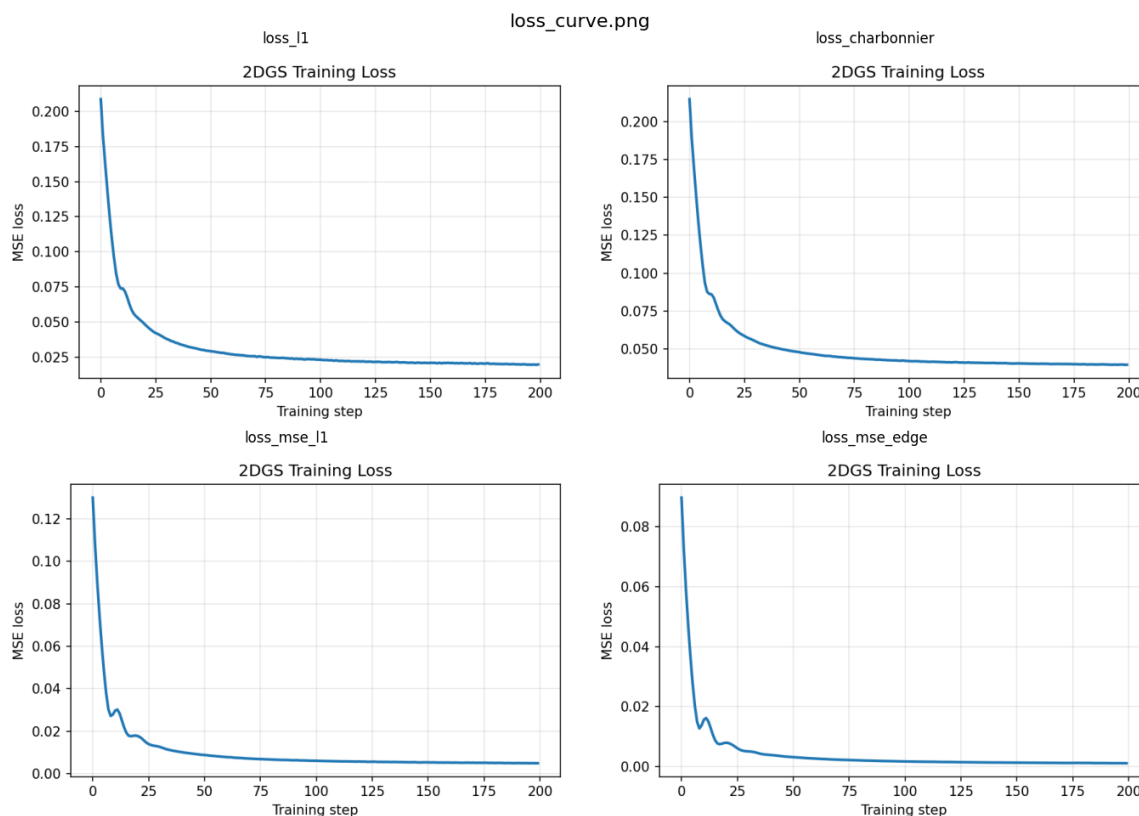
$$PSNR = 10 \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \tag{23}$$

其中:

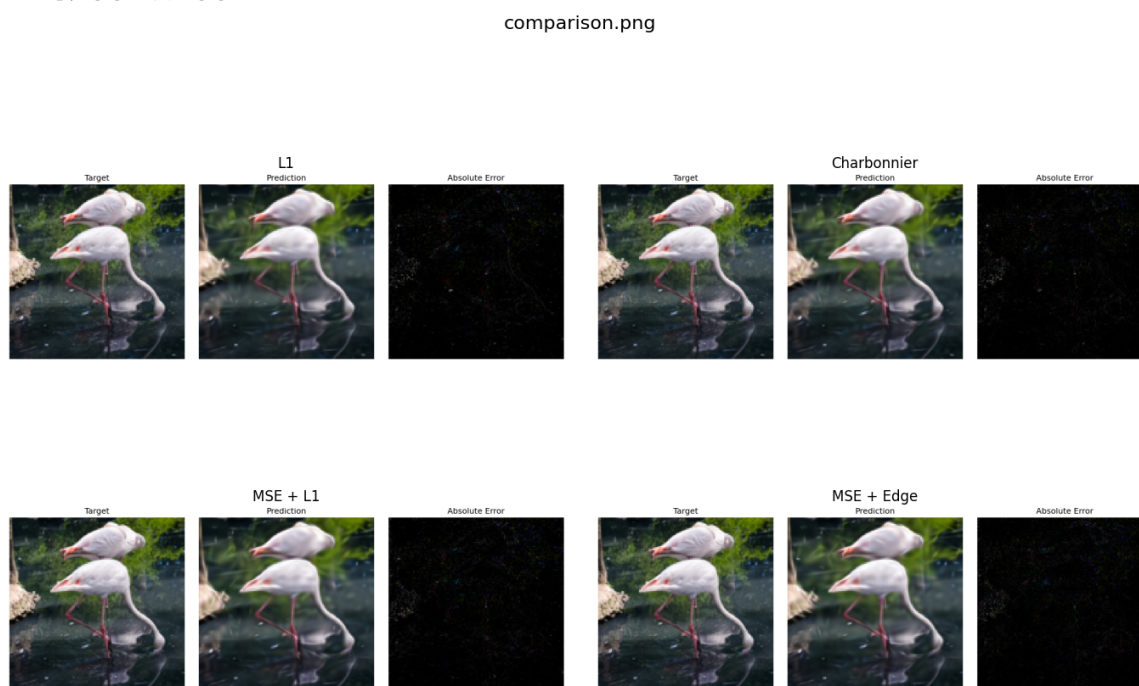
- MAX_I 是像素最大值
- MSE 是均方误差

Loss 模块

Loss 曲线图:



重建对比图与误差图:



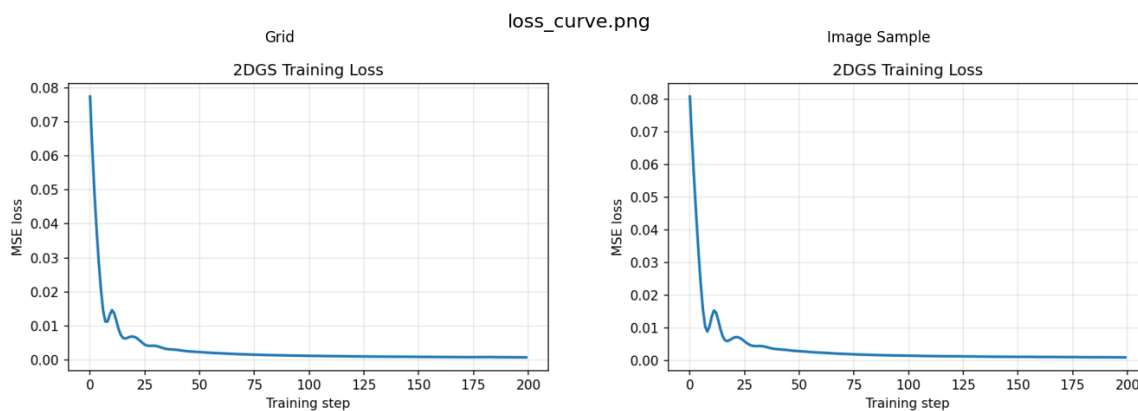
分析讨论:

分析总表的三项指标以及 loss 曲线图, 可以看出新增的四种loss计算方法相比基准实验中的 mse 损失函数都能进一步地去降低平方绝对误差。其中 mse_edge_loss 方法具有更快的收敛速度, 且三项指标都优于基准实验的结果。

而从重构对比图以及误差图中可以看出, 五种损失函数在最终视觉效果上其实差距并不是很大。

Initializers 模块

Loss 曲线图：



重构对比图与误差图：



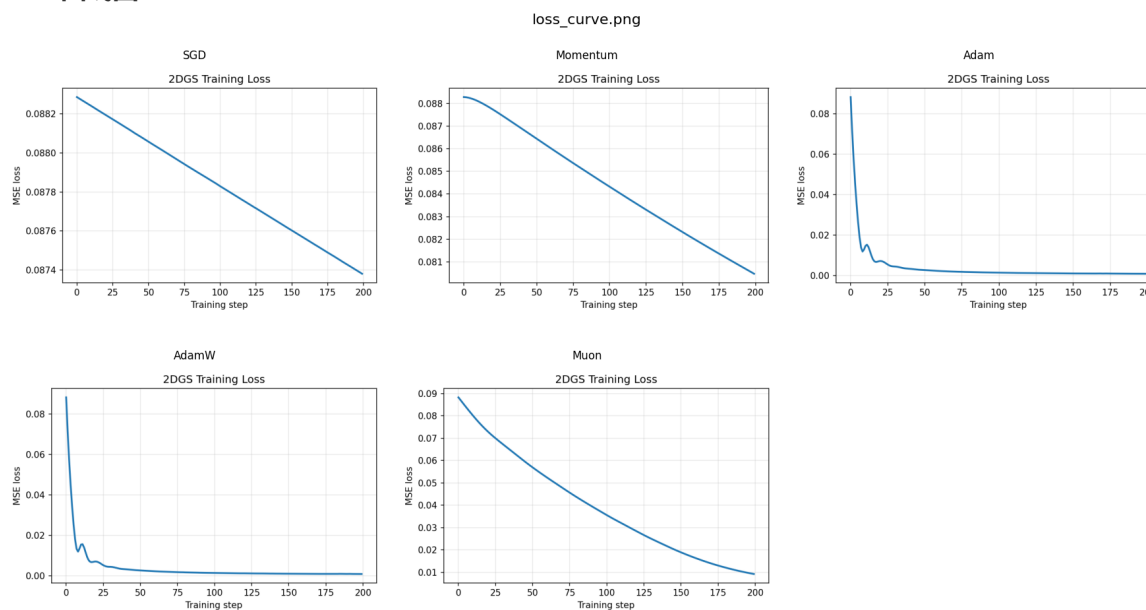
分析讨论：

分析总表的三项指标以及 loss 曲线图，可以看出 Grid 初始化方法在最终结果的三个指标中都是更优于 image_sample 初始化方法的，也优于基准实验的方法。但是从前50步收敛速度上来看，image_sample 初始化的收敛速度会略快于 Grid 初始化方法，两种初始化方法的收敛速度都优于基准实验。

而从重构对比图以及误差图中可以看出，两种初始化方法在最终视觉效果上其实差距并不是很大，Grid 初始化方法拟合的图像包含的细节相比 image_sample 而言似乎会略微多一点。

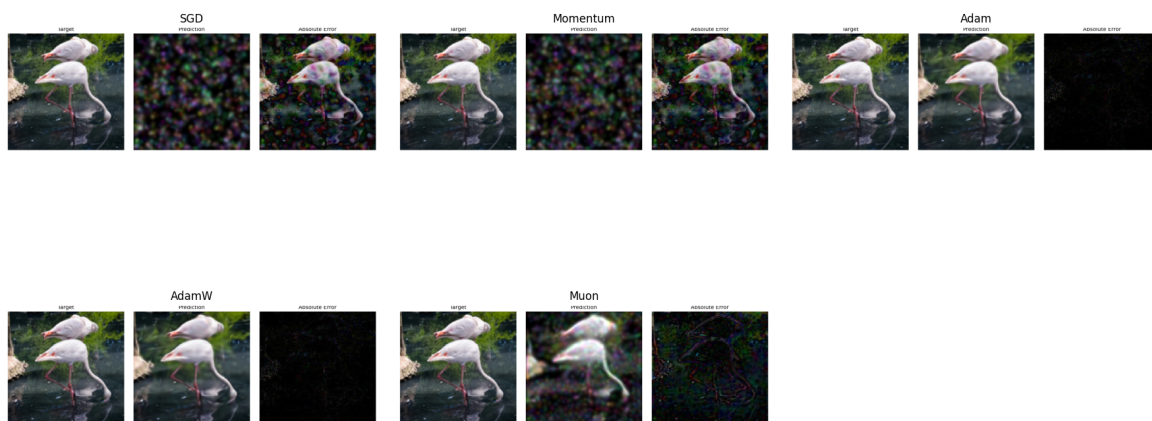
Optimizers 模块

Loss 曲线图：



重构对比图与误差图：

comparison.png



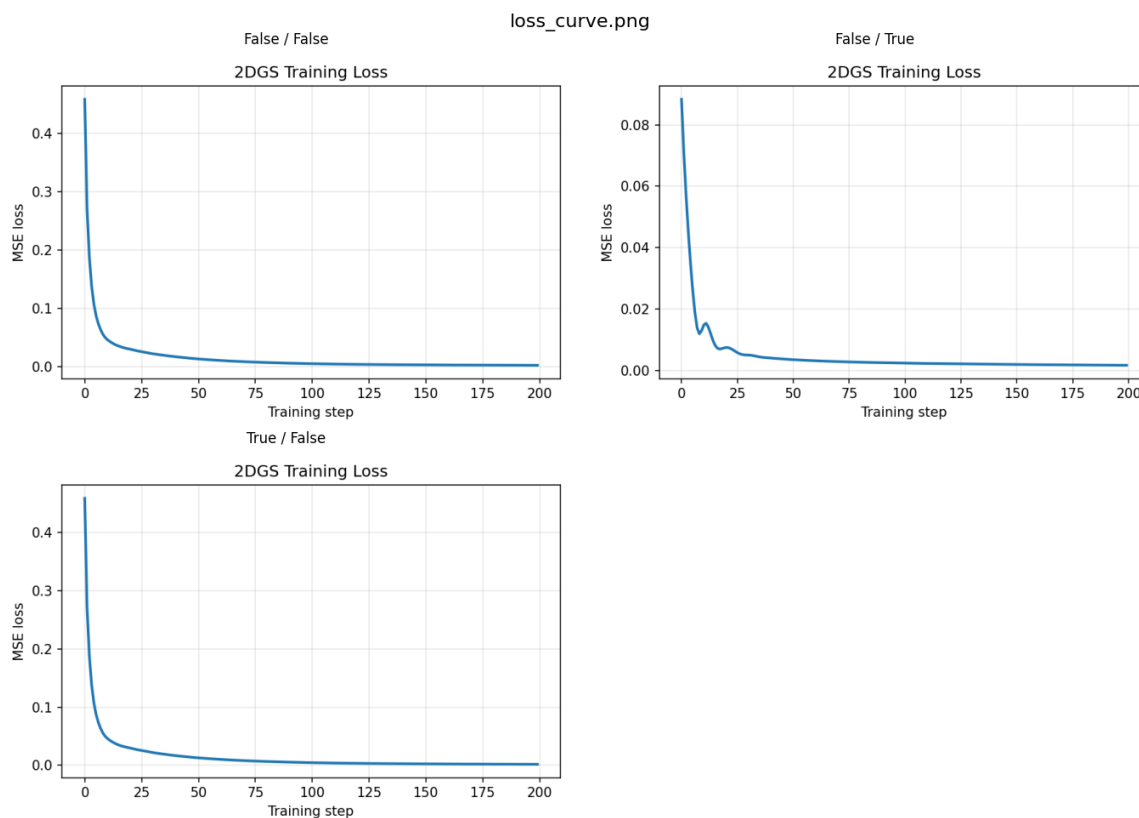
分析讨论：

首先分析总表的三项指标以及Loss曲线图，可以看到 `sgd`、`momentum` 的效果极差，收敛速度慢；`muon` 的效果也不佳，收敛速度较慢。而 `adam` 以及 `adamw` 的收敛速度都较快，且 `adam` 的效果比基准实验中的 `torch_adam` 的效果还要略好一点，这个原因可能是优于自己实现的 `adam` 简化了一些操作，意外地在这张图片取得更好的效果。

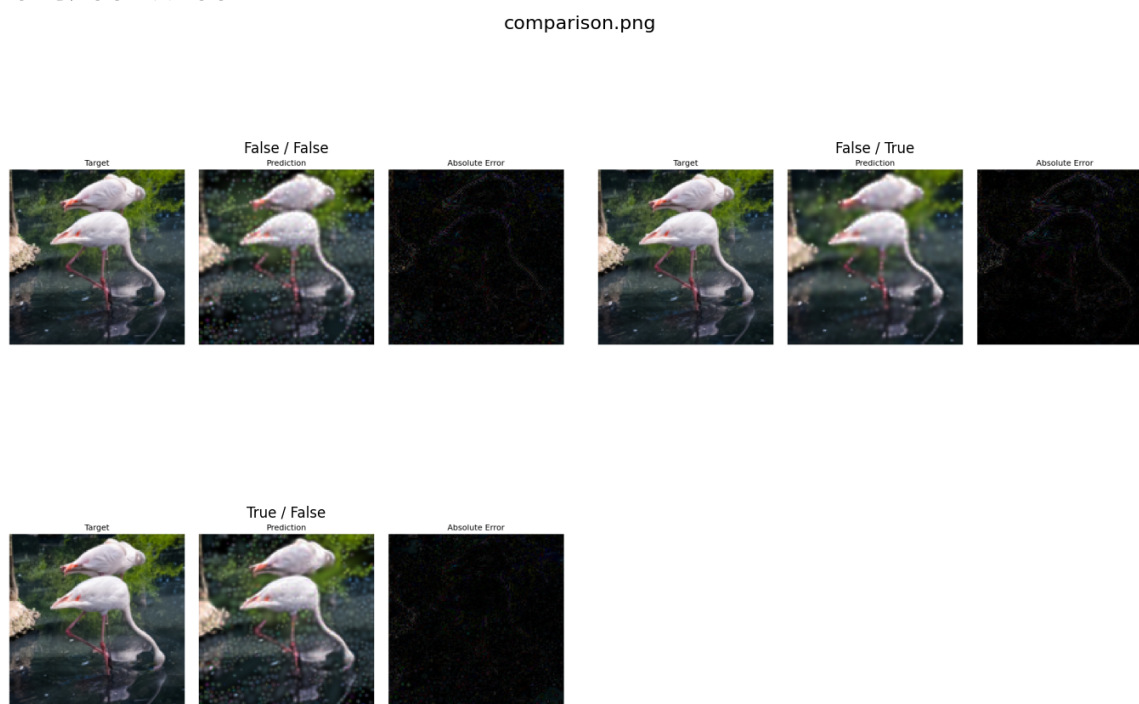
- 对于 `sgd` 和 `momentum` 表现较差，从loss曲线图来看，主要是拟合速度过慢了。这个原因可能是 `sgd` 方法中是所有参数共用一个学习率，但是对于高斯散点图中不同的参数，例如 `center`、`sigma`、`color` 等参数的值域大小、调节幅度每次都应该是不一样的，共用一个学习率会导致参数变化幅度一致，可能改好一些参数后，把别的已经较好的参数改差了。`momentum` 中引入了速度和动量的概念，使得当前方向的优化步长会受历史方向影响，因此对纯 `sgd` 方法是有所提升的，在loss曲线图中可以看到200 step时的损失值会比纯 `sgd` 方法要低上一点。而 `adam` 是在 `momentum` 的基础上，进一步引入了二阶动量，并将优化公式改为 $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$ ，此处不再是每种参数都是一致的学习率，而是会乘上 $\frac{1}{\sqrt{\hat{v}_t}}$ ，使得每个参数有符合自己特点的学习率，因此能取得更好的结果。
- `muon` 这个主要是应用在大模型训练中的优化方法，它更关注与参数更新的方向是结构稳定的，从而避免训练不稳定，梯度病态，深层表示优化等问题。不具有 `adam` 如此的快速拟合速度。而在稳定性方面，确实可以看到 `adam` 和 `adamw` 在拟合过程中会出现一个小的损失反弹峰，而这个在 `muon` 的损失图像中是没有的，这也反映出了 `muon` 的稳定性是要比 `adam` 更好的。

Model 模块

Loss 曲线图:



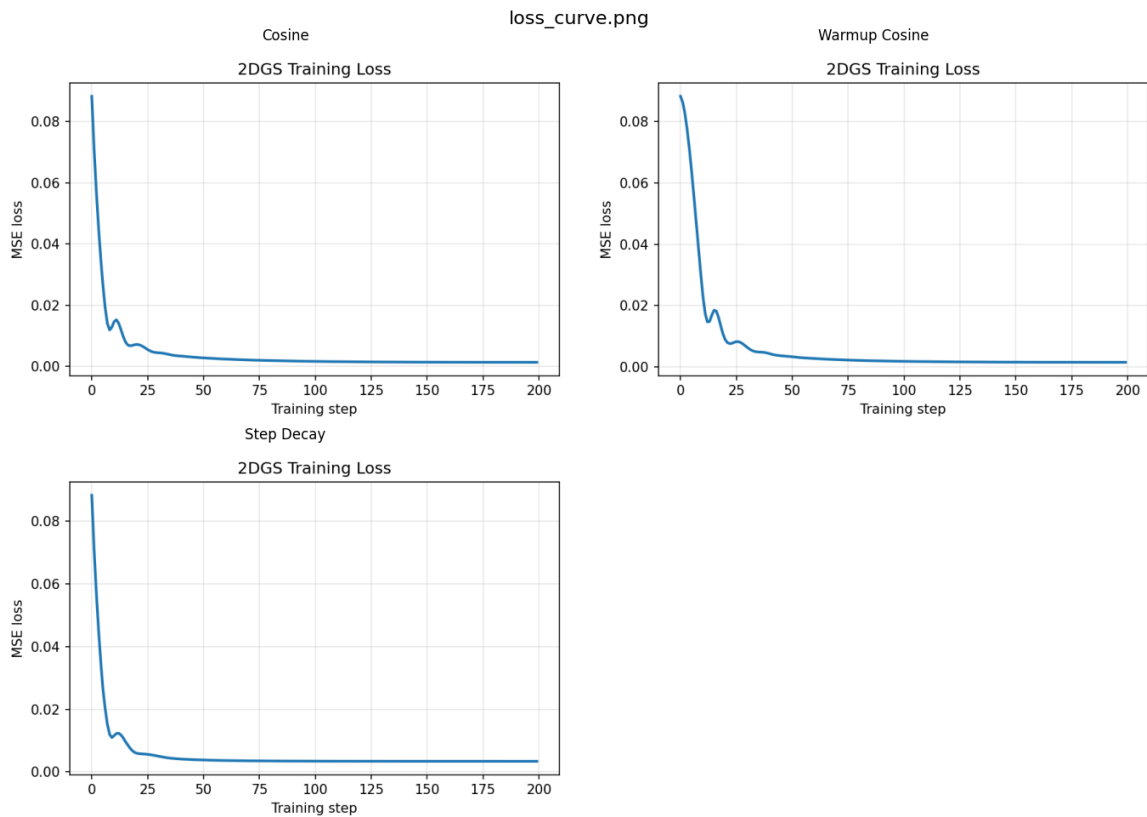
重构对比图与误差图:



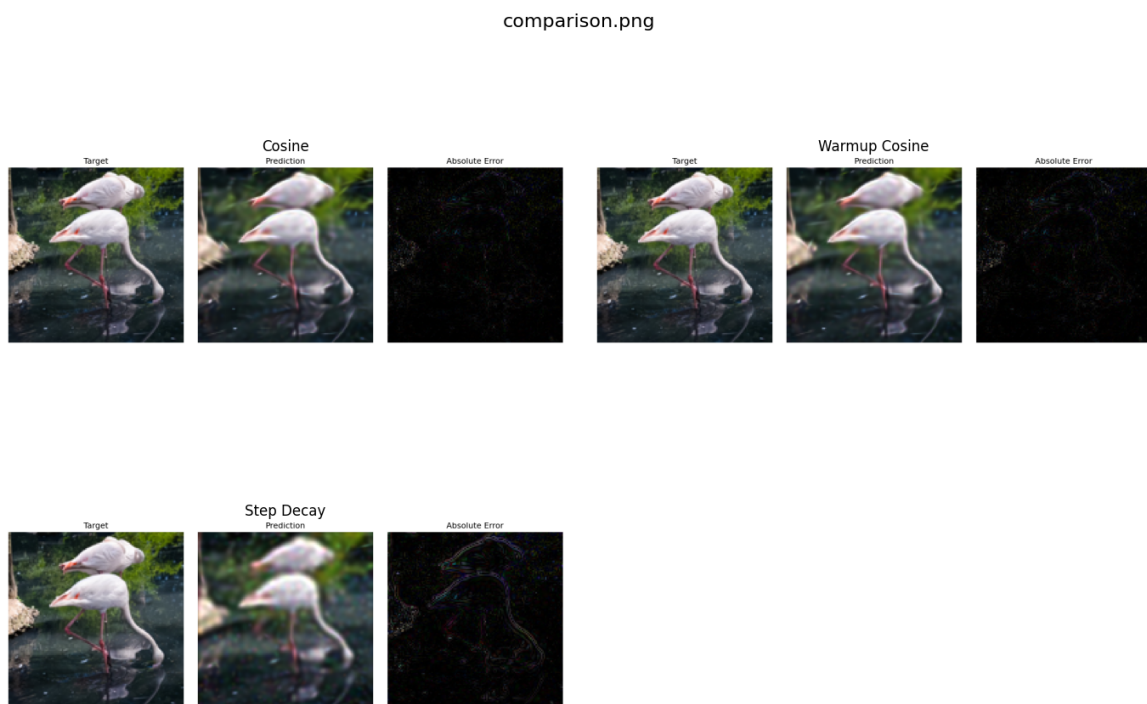
分析讨论: 由于调节的参数是是否使用各项异性以及每个高斯点是否具有自己的透明度参数。显然，一个图像的每一团色块不可能都是正好一个均匀的圆点，也会存在比较透明的地方。因此两个参数一定是 True-True 的时候，效果是最佳的，而实验的结果也正是如此。其中进一步会发现，单独使用各向异性的效果要比单独使用独立透明度的效果更好，表现在PSNR指标上。且二者同时使用会有 $1 + 1 > 2$ 的效果。此外从loss曲线图中可以看出，似乎开启了每个独立透明度参数后，会导致损失曲线有一个小波动。

Scheduler 模块

Loss 曲线图:



重构对比图与误差图:



分析讨论: 从三个指标的总表以及loss曲线图可以看出以下几点:

- 拟合速度: `step_decay > cosine > constant > warmup_cosine`
- 最终的PSNR: `constant > cosine > warmup_cosine > step_decay`

从loss曲线图中可以切实看出，最终 `step_decay` 并没有收敛到0，而是提前收敛了。从最终的重构图以及误差图也比较明显看出其重构的图像比较糊，误差图中也还有很多明显的点以及曲线。

在深度学习中，不同学习率调度器的最终效果排名可能是这样的：
`warmup_cosine` $\succ$ `cosine` $>$ `step_decay` $>$ `constant` 这个与我们最终PSNR的排行确实有所不同。这个也许与任务的性质有关，在本次任务中我们主要是去拟合一个确定的图像，而不要求具备泛化能力。此外考虑到不同学习率调度器的特点，也有可能是本任务中有许多较小的局部极小值点，学习率会逐渐下降的算法都容易提前陷入局部极小值点，而 `constant` 由于学习率不变，因此更有可能跳出这些局部极小值点。

任务2

任务2A

模块配置：

项目	图像大小	高斯数量	步数	Loss	Optimizer	lr	Scheduler	Initializer	各向异性	Alpha	随机种子
默认配置	128 × 128	1000	100	mse_edge	student_adam	6.7e-2	constant	importanceTaskA	True	True	42

实验结果：

```
1  === Task2A (100 steps) ===
2  R1_flamingo           PSNR = 29.9838 dB
3  R2_starry_night       PSNR = 27.1139 dB
4  R3_parkour            PSNR = 28.6212 dB
5  S1_night_cityscape    PSNR = 28.9840 dB
6  S2_mandala            PSNR = 32.7977 dB
7  S3_coral_reef         PSNR = 30.0790 dB
8  AVERAGE              PSNR = 29.5966 dB
```

分析：

`Importance` 初始化是采用了提示中思路1的重要性采样，将原图转换为灰度图，计算梯度强度 `gray_mag`，并采用均值池化，计算局部方差 `local_var` 以及边缘邻域强度 `edge_strength`，然后对这三个因素进行加权求和，作为高斯半径点在图中的采样概率，从而实现重要性采样。

`importanceTaskA` 是在 `importance`` 初始化的基础上，加上了提示中提及的思路二。通过计算每个高斯的最近邻的平均距离，使采样更密的高斯半径更小，采样稀疏部分的高斯半径更大。

`cosineTaskA` 是在 `cosine` 的基础上，为其加入一个返回值下界。

由于实验A是一个追求快速收敛的实验，因此我采用了在任务一中测试出效果最好也是收敛较快的 `mse_edge` 以及 `student_adam`，搭配上 `importanceTaskA` 相较于基准测试有明显提升。然后简单调节一下初始学习率便能取得以上效果。

实验：

实验基准配置为：

项目	图像大小	高斯数量	步数	Loss	Optimizer	lr	Scheduler	Initializer	各向异性	Alpha	随机种子
基准配置	128×128	1000	100	mse	torch_adam	5e-2	constant	random	True	True	42

实验观察：

设计	对照	收益
基准配置	27.2515	/
采用 mse_edge、grid、student_adam	27.2515->27.8176	+0.5661
采用 importance	27.8176->29.1578	+1.3402
学习率改为 11e-2，调度器改为 cosine_task2A	29.1578->29.2157	+0.0579
学习率改为 6.48e-2，调度器改为 constant	29.1578->29.3479	+0.1901
采用 importanceTaskA	29.3479->29.4584	+0.1105
学习率改为 6.7e-2	29.4584->29.5966	+0.1382

其中还有多次学习率的测试，就不再此列出。实验中发现 importance 以及 importanceTaskA 初始化对PSNR大概能有1.5的提升，此外对学习率的微调也能有不错的提升。

任务2B

模块配置：

项目	图像大小	高斯数量	步数	Loss	Optimizer	lr	Scheduler	Initializer	各向异性	Alpha	随机种子
默认配置	128×128	1000	500	mse_edge	student_adam	3e-2	constant	importanceTaskA	True	True	42

实验结果：

1	=== Task2B (500 steps) ===										
2	R1_flamingo	PSNR = 32.6730 dB									
3	R2_starry_night	PSNR = 28.8890 dB									
4	R3_parkour	PSNR = 32.6222 dB									
5	S1_night_cityscape	PSNR = 34.0142 dB									
6	S2_mandala	PSNR = 39.9840 dB									
7	S3_coral_reef	PSNR = 39.0004 dB									
8	AVERAGE	PSNR = 34.5305 dB									

分析：

由于任务B是一个追求精度的实验，因此 loss 函数以及 optimizer 都选择了在实验一中表现最好的，模型设计方面也很理所应当的选择了同时开启独立透明度以及各向异性。在学习率调度器方面，可能 cosine、warmup_cosine、step_decay 这三种调度器都有自己的超参数，而可能超参数我没有太调节好，因此结果都不如 constant 的效果好（也可能是实验一中提到这三种调度器不适合这一任务的原因）。故在学习率调度器中我还是选择了 constant，经过简单的初始学习率调整，便能取得平均 PSNR=34.4179的不错表现。

实验：

实验基准配置为：

项目	图像大小	高斯数量	步数	Loss	Optimizer	lr	Scheduler	Initializer	各向异性	Alpha	随机种子
基准配置	128 × 128	1000	500	mse	torch_adam	5e-2	constant	random	True	True	42

实验观察：

设计	对照	收益
基准配置	31.5370	
采用 mse_edge、grid、student_adam	31.5370->31.9234	+0.3864
初始化改为 importance、调度器使用 warmup_cosine	31.9234->33.3032	+1.3798
调度器使用 constant	33.3032->32.9848	-0.3184
学习率调节为 3e-2	32.9848->34.4179	+1.4331
初始化采用 importanceTaskA	34.4179->34.5305	+0.1126

实验中发现，主要收益来自于 importance 和 importanceTaskA 初始化方法的选择，以及学习率从 5e-2 调节到 3e-2 也能有十分明显的提升，这也比较符合任务B的目的也许，毕竟是一个在多step下，去追求更优结果，学习率下降搭配上costant确实理论上后期的收敛会更加。